

Wie funktioniert KI?

Von Dense Layern zu Garbentheorie

Überblick

Dense Layer – die Grundbausteine

Positional Encoding

Der große Überblick – Token-Vorhersage

Was passiert in den Layern?

Raumkrümmung

Topologie

Holografische Kodierung

Garbentheorie

Zusammenfassung

Dense Layer – die Grundbausteine

Schritt 1

Dense Layer – die Grundbausteine

Dense Layer – Grundidee

$$f(x) = Ax + b$$

- › A = Gewichtsmatrix (Tensor), b = Bias-Vektor
- › Jeder Eingang mit jedem Ausgang verbunden
- › Lineare Transformation: Drehen, Strecken, Verschieben



*Bild: Dense Layer –
Knoten voll verbunden*

Verschachtelung – warum reicht *eine* Schicht nicht?

Zwei Dense Layer hintereinander:

$$g(f(x)) = A_2 (A_1 x + b_1) + b_2 = \underbrace{A_2 A_1}_{= A'} x + \underbrace{A_2 b_1 + b_2}_{= b'}$$

Kollabiert zu einer einzigen linearen Abbildung!

- Egal wie viele Schichten $\rightarrow$ bleibt $f(x) = A'x + b'$
- Tiefe bringt **nichts** ohne Nichtlinearität

Aktivierungsfunktionen – der Trick

$$h(x) = \sigma(Ax + b)$$

- σ = nichtlineare Funktion (ReLU, Sigmoid, Tanh, ...)
- Bricht die Linearität auf
- Jetzt: $g(\sigma(f(x))) \neq A'x + b'$

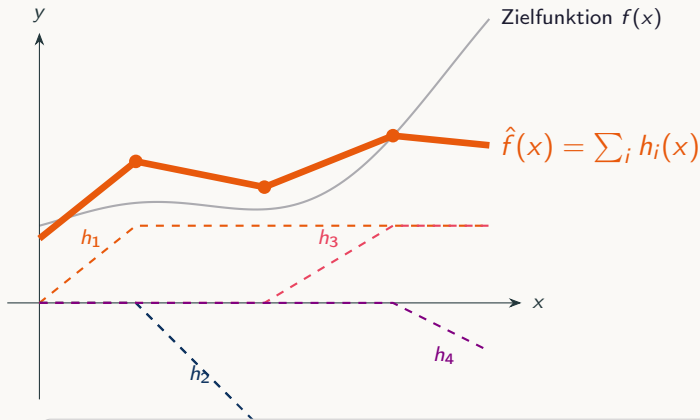
Dense + Nichtlinearität + Dense

$\Rightarrow$ *universeller Approximator*

(Cybenko 1989, Hornik 1991)



Wie approximiert ein Netz eine Funktion?



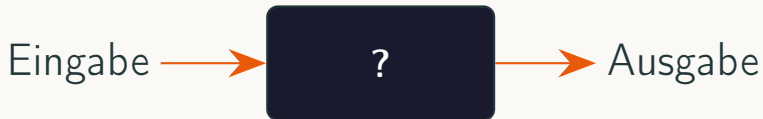
Jedes Neuron $h_i(x) = a_i \max(0, x - c_i)$ liefert einen **Knick** an Position c_i .

4 Neuronen $\rightarrow$ **3 Knicke** $\rightarrow$ grobe Approximation.

100 Neuronen $\rightarrow$ **99 Knicke** $\rightarrow$ fast beliebig genau.

Negative Steigung entsteht durch die zwei Dense-Schicht, die wieder negative Gewichte haben kann

Black Box – wir sehen nur Input & Output



- › Netzwerk lernt *interne Muster* – wir sehen sie nicht direkt
- › Kann jedes stetige Problem approximieren ...
- › ... versteht aber nicht *warum*

Beispiel: Gartenbewässerungs-KI

Ziel: Ernte maximieren

Eingabe: Bewässerungszeitpunkte

Zeitfenster	Erdbeeren (kg)	Kartoffeln (kg)	Tomaten (kg)	Zucchini (kg)
06:00 – 08:00	12,4	38,1	9,7	15,2
12:00 – 14:00	7,1	33,5	5,2	12,8
18:00 – 20:00	11,9	37,4	9,3	14,9
23:00 – 02:00	4,3	28,6	4,1	9,7
Optimal (KI)	13,1	39,2	10,4	16,0

Versteckte Zusammenhänge

› Nachts wässern?

Schnecken aktiver bei Nässe → fressen Ernte ab

› Wind egal?

Mehr Wind → mehr Verdunstung, schlechtere Aufnahme

› Mittags wässern?

Wassertropfen = Linsen → Sonnenbrand auf Blättern

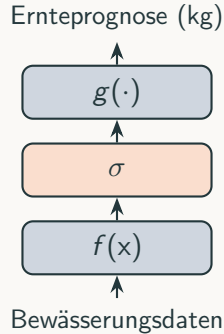
→ KI lernt: “nachts = schlechter Ertrag”

Weiß aber nicht, dass Schnecken der Grund



Bild: Garten + Schnecke

Verschachtelte Layer = Abstraktion



jetzt Bewässern ja/nein = $g(\sigma(f([\text{Temperatur, aktuelle Uhrzeit, Windgeschwindigkeit, \dots]})))$

Positional Encoding

Schritt 2

Positional Encoding

Positional Encoding – Wo steht ein Wort?

$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

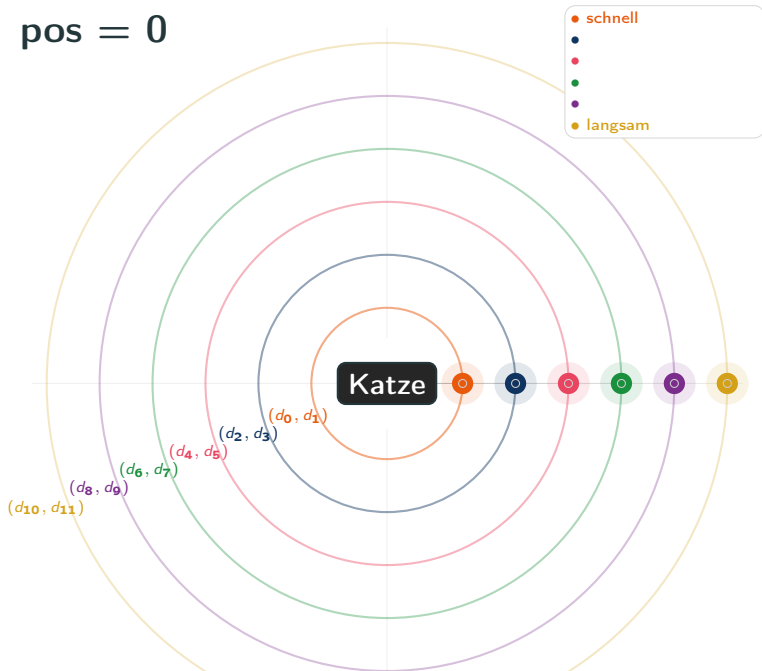
- › Jede Position bekommt ein **einzigartiges Muster** aus Sinus-/Cosinus-Wellen
- › Wird auf den Wortvektor **addiert** (nicht konkateniert)
- › Hochdimensionale “Spirale” um jedes Wort
- › Wort $\neq$ einzelner Punkt, sondern eine **Gegend** im Raum
- › Relative Positionen $\rightarrow$ Beziehungen zwischen Wörtern

Positional Encoding – Visualisierung

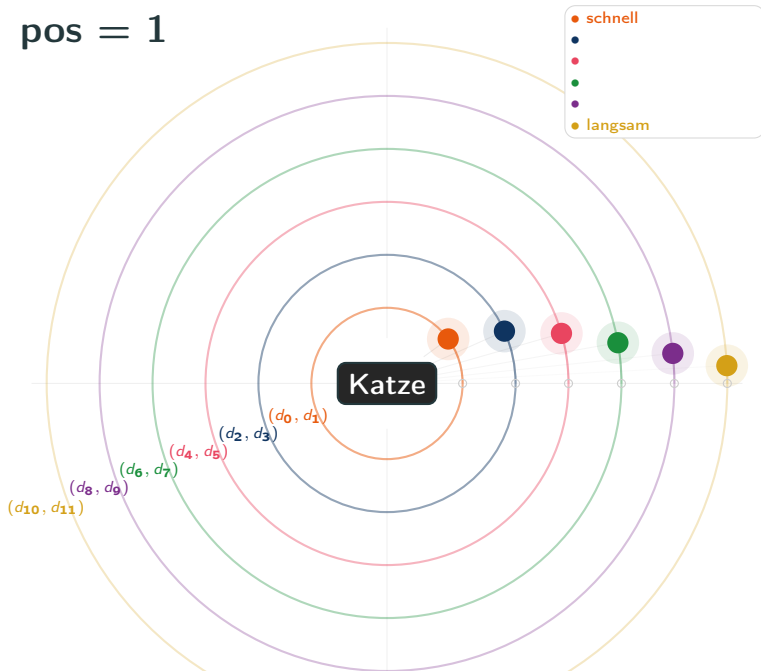


*Bild: hochdimensionale Spirale / PE-Heatmap
(vorhanden – hier einfügen)*

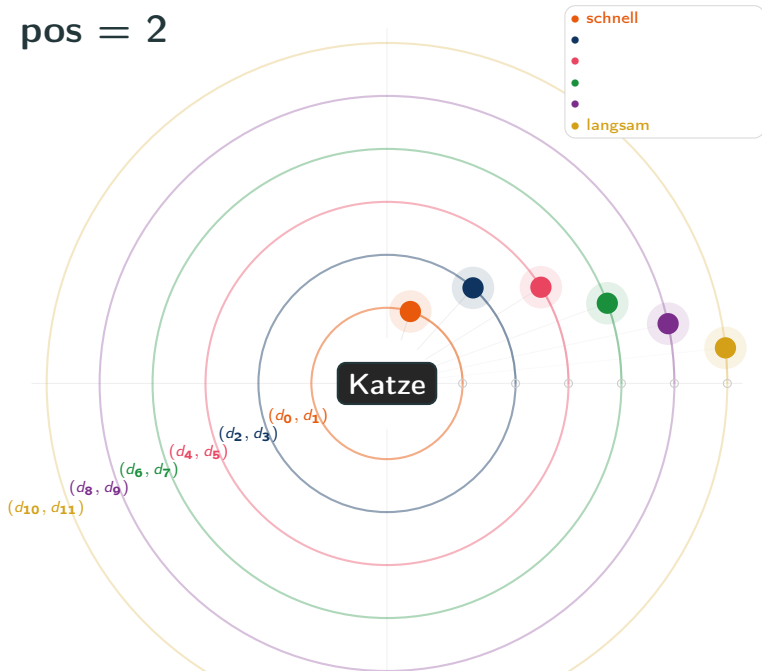
pos = 0



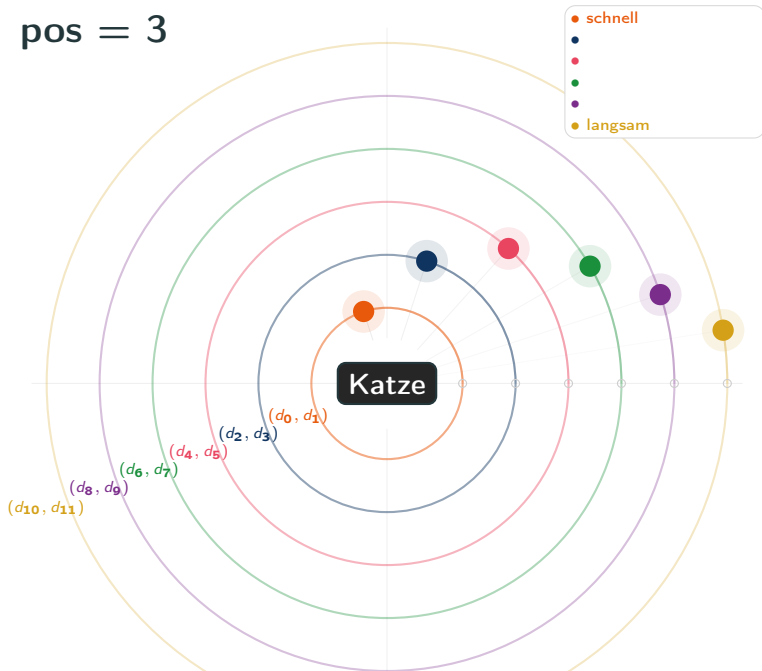
pos = 1



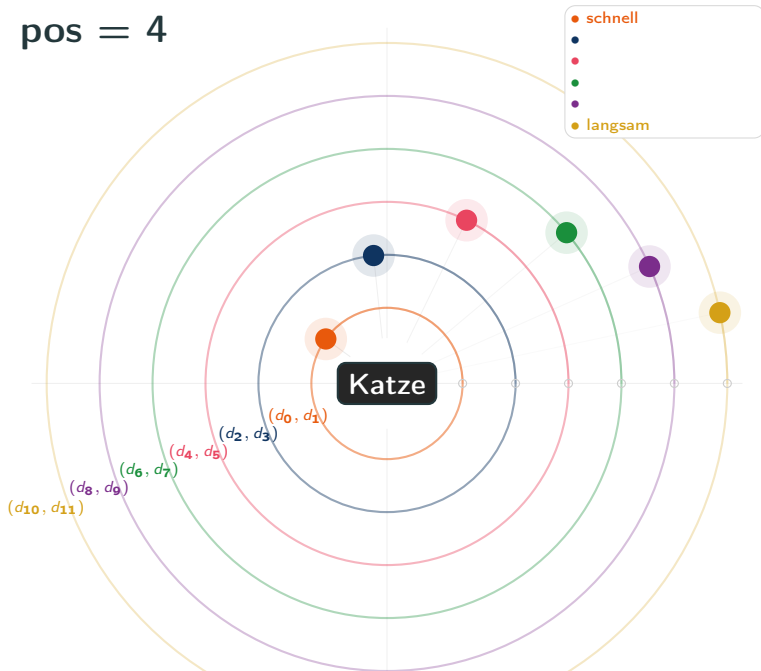
pos = 2



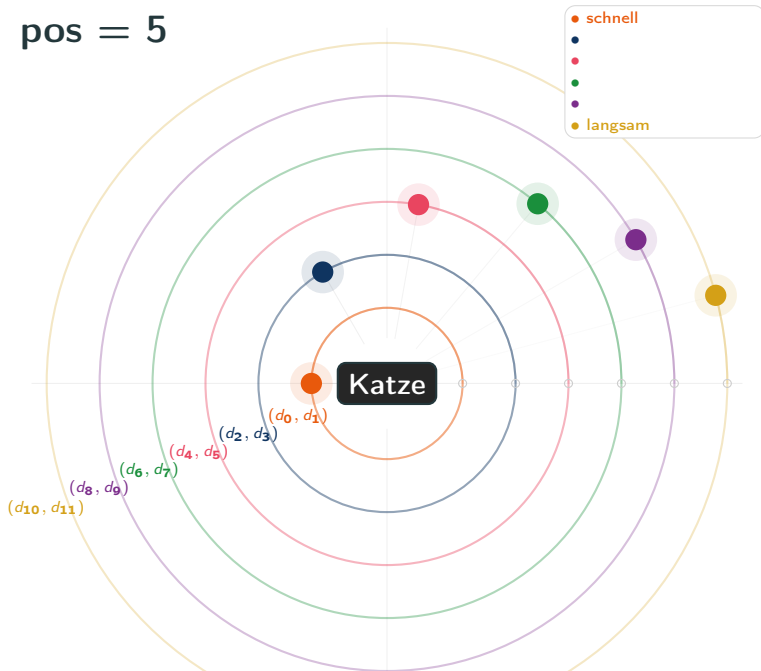
pos = 3



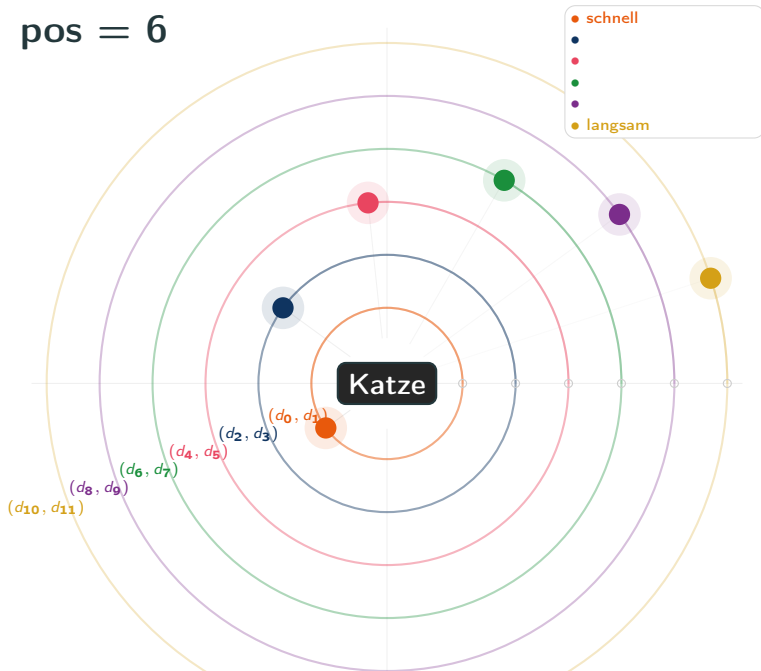
pos = 4



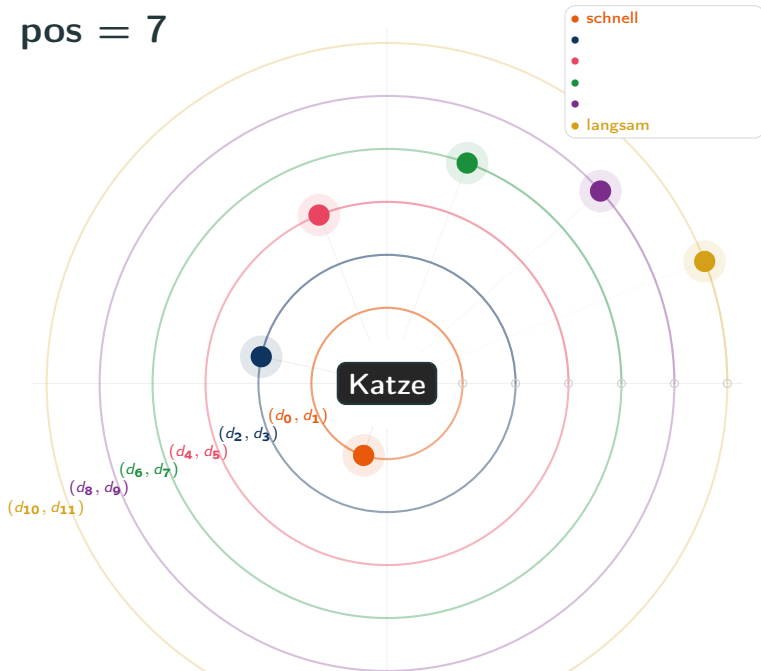
pos = 5



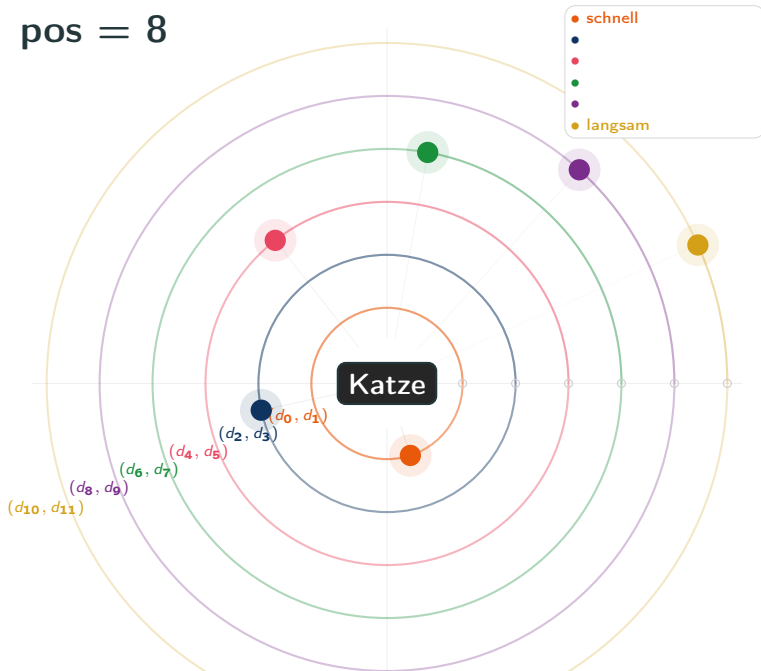
pos = 6



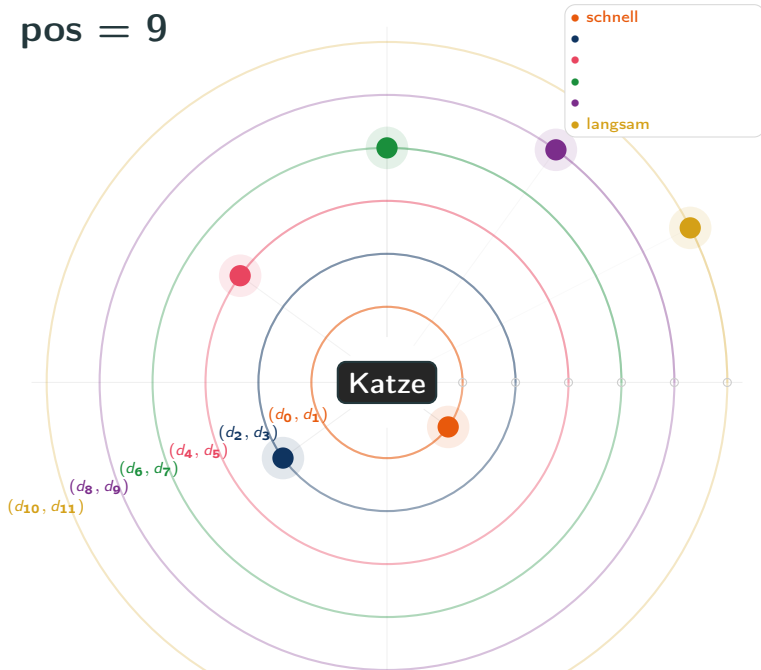
pos = 7



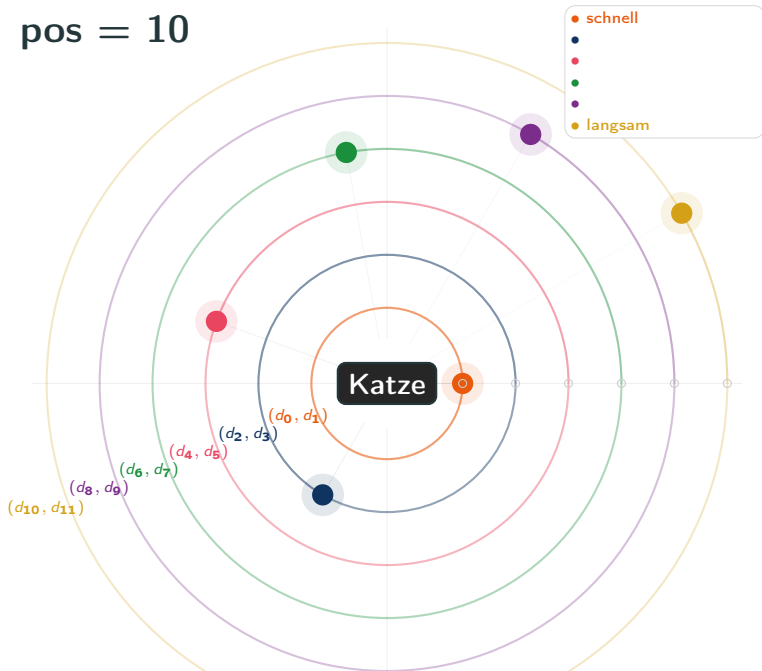
pos = 8



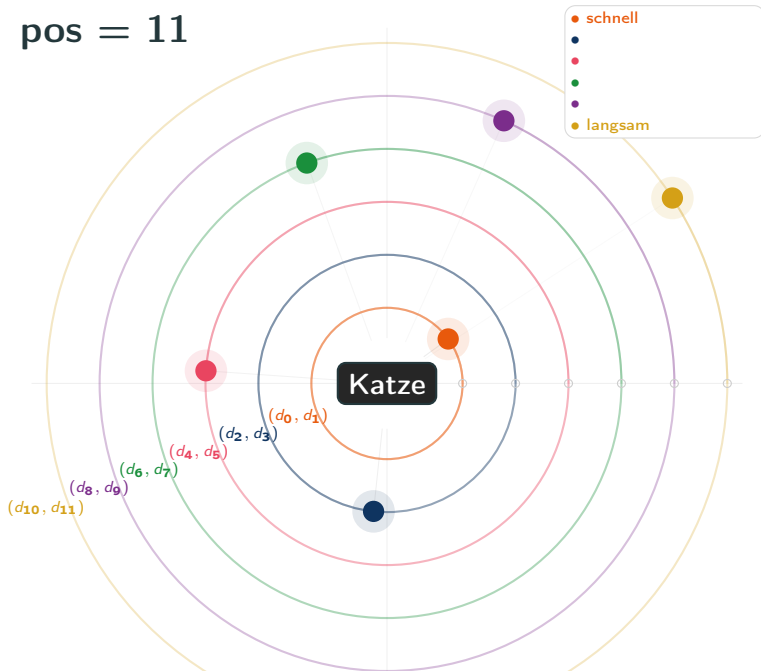
pos = 9



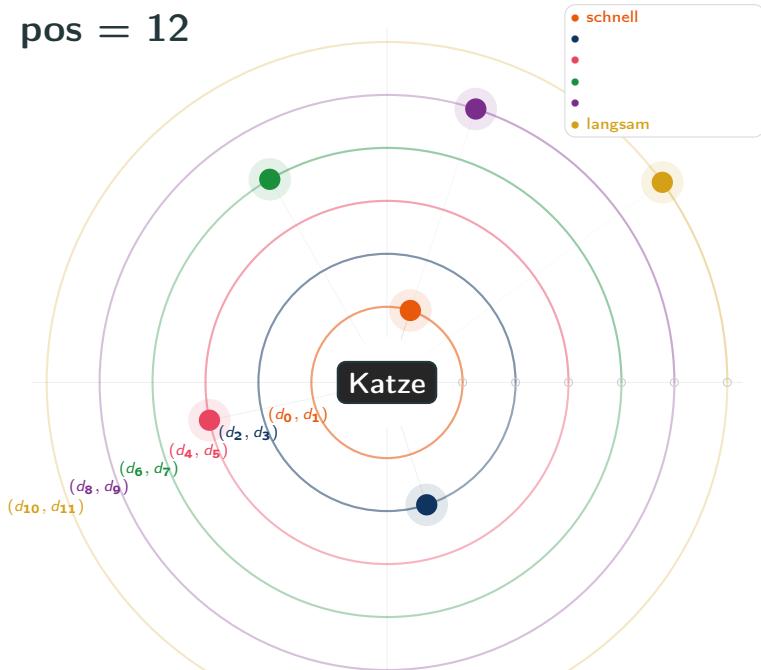
pos = 10



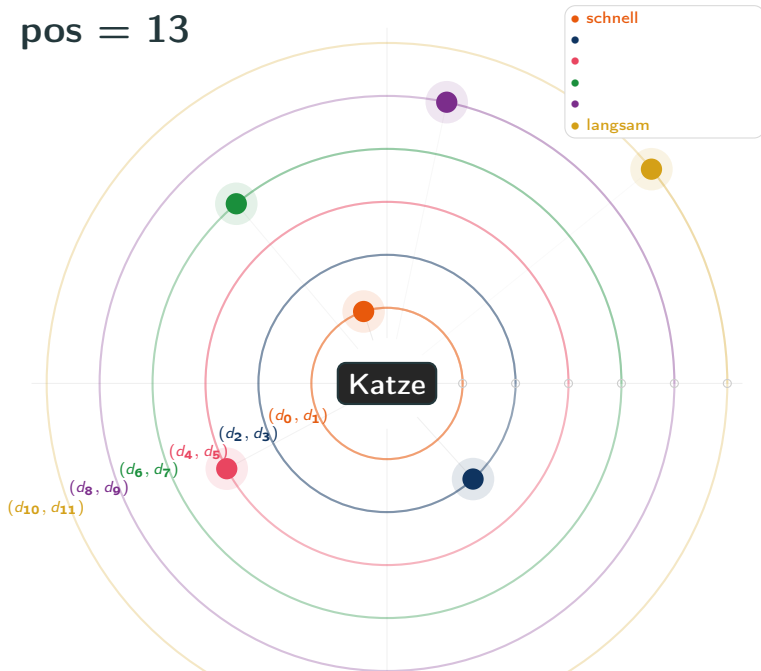
pos = 11



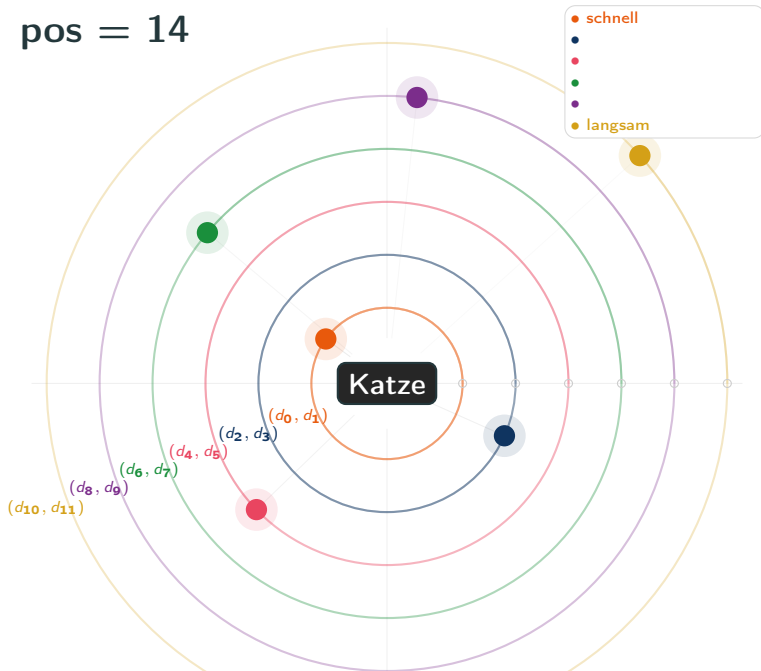
pos = 12



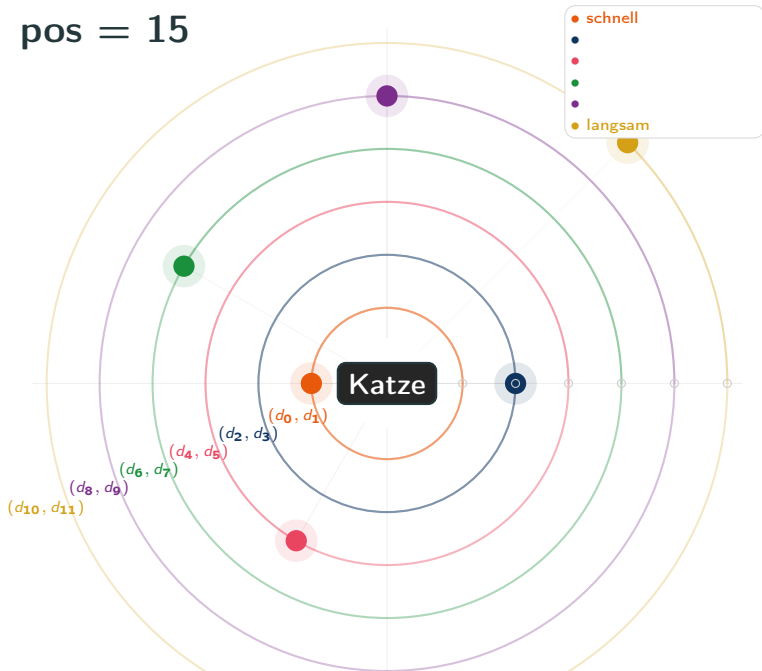
pos = 13



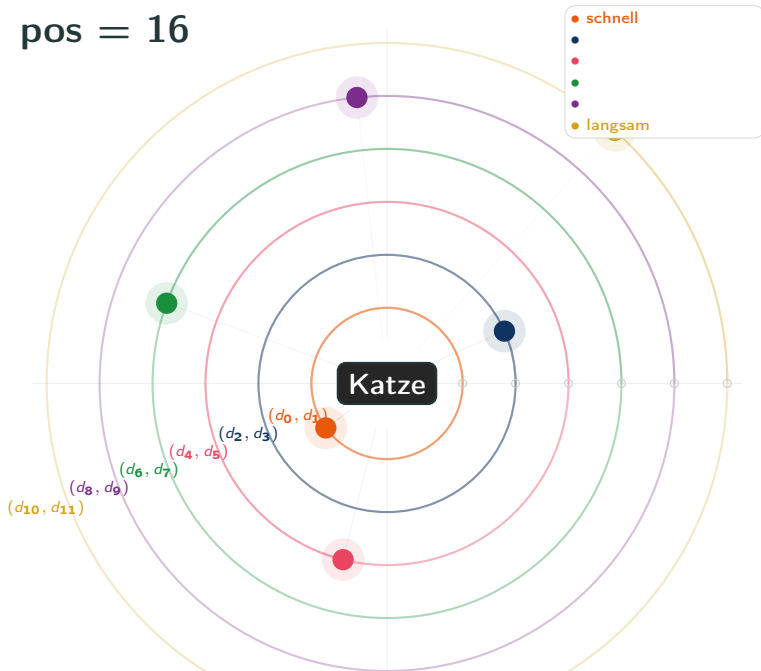
pos = 14



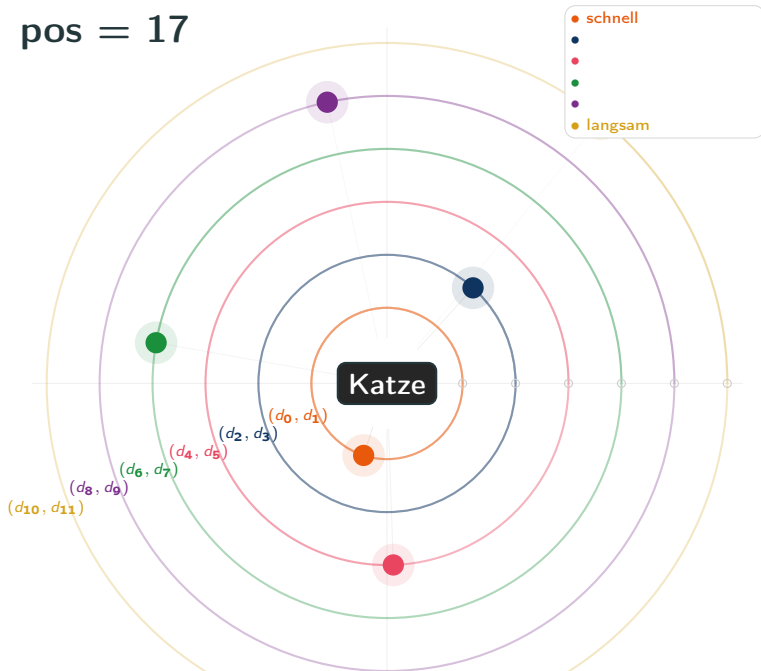
pos = 15



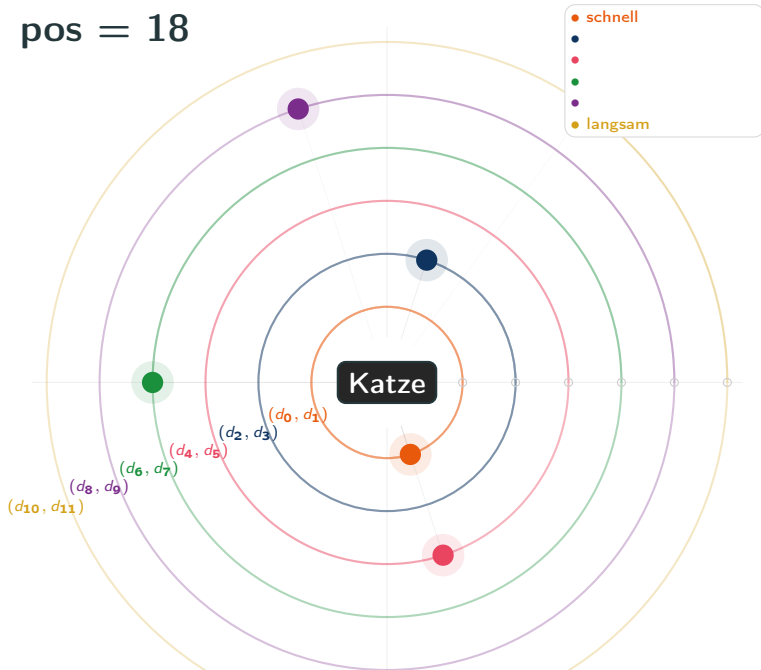
pos = 16



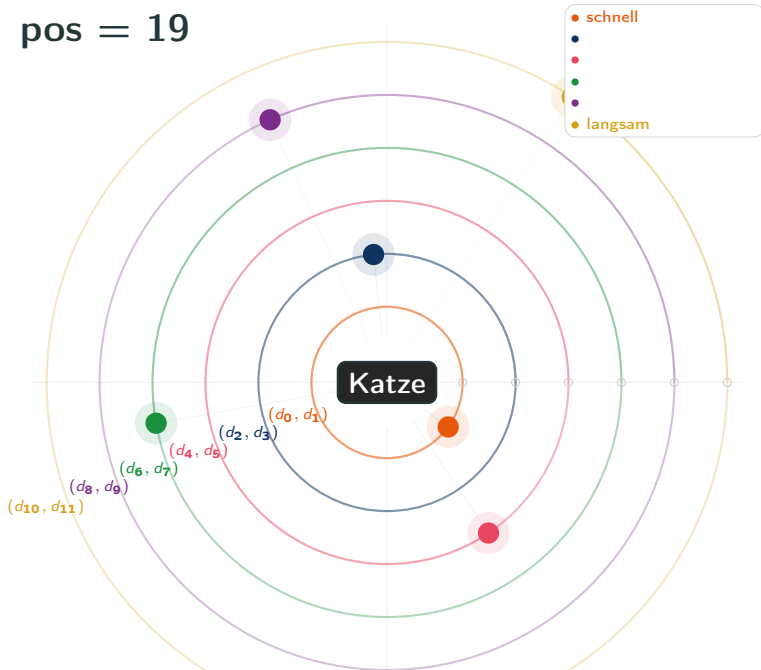
pos = 17



pos = 18



pos = 19

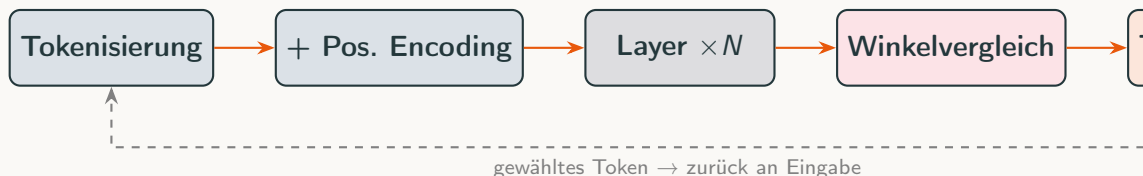


Der große Überblick – Token-Vorhersage

Schritt 3a

Das Gesamtbild – Token-Vorhersage

Wie ein Sprachmodell Text generiert



- › Layer passen **Winkel** zwischen Token-Vektoren an
- › Am Ende: Winkel zu *allen* bekannten Tokens vergleichen
- › Top- k Kandidaten → **zufällig** einen wählen
- › Gewähltes Token wird angehängt → Prozess wiederholen

Temperatur – Kreativität vs. Sachlichkeit

- $T \rightarrow 0$: immer das wahrscheinlichste Token → **sachlich, deterministisch**
- $T \rightarrow \infty$: gleichverteilte Wahl → **kreativ, chaotisch**
- Name: **Boltzmann-Verteilung**
Ursprünglich: Geschwindigkeitsverteilung von Gasmolekülen



*Bild: Boltzmann +
Gasmoleküle*

Wann hört die KI auf?

Frage: Was ist $2+2$?

Antwort: 4 <eos>

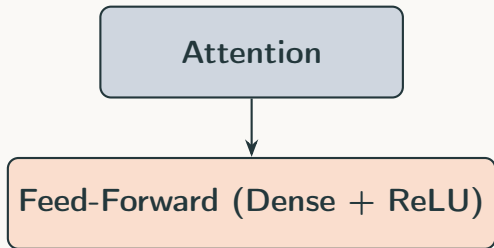
- <eos> = *End of Sequence* – spezielles Token
- Genauso gelernt wie alle anderen Tokens
- In Trainingsdaten stand es am Ende jeder Antwort
- Wenn das Modell <eos> als nächstes Token wählt → **Generierung stoppt**

Was passiert in den Layern?

Schritt 3b

Was passiert *in* den Layern?

Zwei Operationen pro Layer

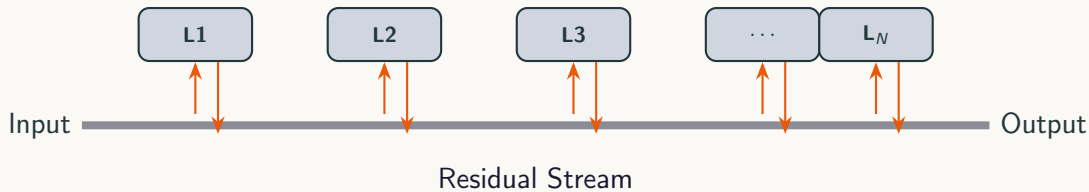


“In welcher **Beziehung** steht jedes Wort zu allen anderen?”

“Was mache ich jetzt mit dieser Info + den Positionen?”

- **Frühe Layer:** konkret – Wortteile zusammenfügen (z. B. brave + 1y → Adverb)
- **Späte Layer:** abstrakt – semantische Zusammenhänge, kaum noch menschlich interpretierbar

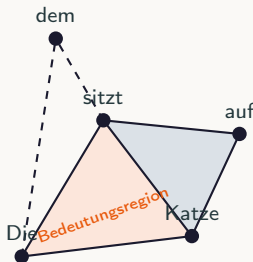
Der Residual Stream – globales Notizbuch



- Ursprünglich erfunden: tiefe Netze besser trainierbar machen (ResNets)
- Hier: **globales Workbook** – jede Schicht liest und schreibt darauf
- Wie ein Text, der einer Reihe von Experten gegeben wird – jeder kritzelt seine Notizen drauf
- Hunderte Schichten, hunderte Experten-Runden

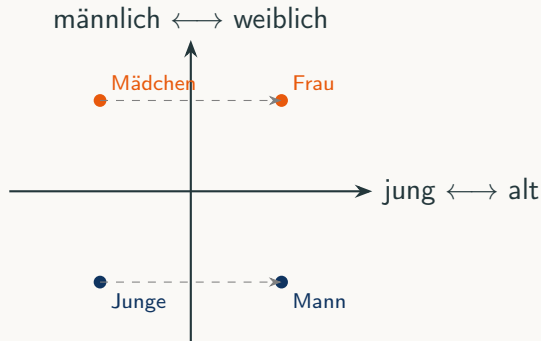
Attention Heads – die Linsen

- Jeder Attention Head = eine **Linse**, die auf einen bestimmten Aspekt fokussiert
- Baut **Beziehungsstruktur** zwischen Wörtern auf
- Ergebnis → an Feed-Forward-Netz (Dense + ReLU)
- FFN entscheidet: *wie muss der Raum der nächsten Schicht angepasst werden?*



Simplicialkomplex: Beziehungen zwischen Tokens als geometrische Struktur – Bedeutung wird “eingekringelt”.

Dimensionen des Embedding-Raums



- Manche Dimensionen: **interpretierbar** (Alter, Geschlecht, Größe ...)
- Andere: **keine menschliche Bezeichnung** – abstrakte Muster
- Modell hat $d = 4096+$ Dimensionen $\rightarrow$ die meisten nicht greifbar

Aber was machen die inneren Layer *wirklich*?



*Vollbild: Faserbündel-Visualisierung mit Fragezeichen
innen (vorhanden – hier einfügen)*

Können neuronale Netze *rechnen*?

$$f(x) = 2x + 5, \quad f(5) = 15$$

- Fakten speichern → klar, ist “nur” Kompression
- Aber: **Algorithmen ausführen?**
- Klappt nicht immer, nicht für alle Algorithmen ...
- ... aber *oft genug*, dass es kein Zufall sein kann
- Wie bildet ein Netz aus Matrixmultiplikationen + ReLU eine *Berechnung* ab?

→ **Meine Forschungsfrage. Hier meine spekulativen Ideen.**

Raumkrümmung

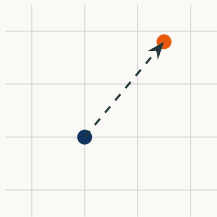
Schritt 4

Raumkrümmung – eine andere Perspektive

Zwei Interpretationen

Interpretation A

Punkte werden im Raum
verschoben



Interpretation B

Der **Raum selbst** wird verformt



- Augenscheinlich identisch – aber:
- Raumkrümmung kann **Information speichern** in der Krümmung selbst
- Und: in der Krümmung kann **gerechnet** werden

Topologie

Schritt 5

Topologie – Geometrie ohne Distanz

Was ist Topologie?

- Geometrie von Oberflächen – aber ohne konkrete Distanzen
- Nur noch: $d(x, y)$ – abstrakte Metrik (im normalen Raum: Pythagoras)
- KI: es geht um **Relationen**, nicht um absolute Abstände
- Daher: **topologische Form** des Embedding-Raums ist entscheidend



Torus (Donut)



Meme: Topologe kann Kaffeetasse nicht von Donut unterscheiden

Loop Spaces – Topologie kann rechnen

- › **Loop Space:** Raum aller geschlossenen Pfade auf einer Fläche
- › Ein Pfad = eine Zahl
Anderer Pfad = andere Zahl
- › Verkettung von Pfaden $\sim$ Addition
- › Umkehrrichtung $\sim$ Subtraktion (Minus)
- › Pfad und Rechnung sind **homotop** – ineinander überführbar

Anwendung: automatisches Beweisen
(Homotopie-Typentheorie)

 *GIF / Bild: Loop Space
beim Zählen – Schleifen
werden verkettet*

Holografische Kodierung

Schritt 6

Holografische Kodierung

Pigeonhole-Prinzip & verteilte Speicherung

- Mehr Konzepte als Neuronen
→ ein Neuron speichert *mehrere* Konzepte
- Kein “Großmutter-Neuron”
(kein einzelnes Neuron = ein Konzept)
- Information **verteilt** über das gesamte Netz
- Wie ein **Hologramm**: jedes Fragment enthält das ganze Bild (unscharf)

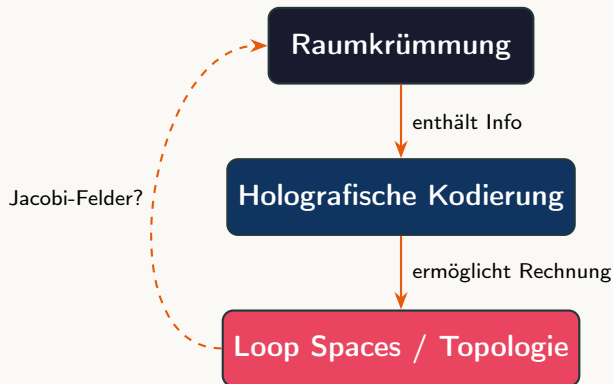


Bild: Pigeonhole-Prinzip



Bild: Hologramm – Smiley links, Hologramm-Muster rechts

Raumkrümmung + Holografie = Berechnung?



- **Hypothese:** Die holografisch kodierte Berechnung findet *in* der Raumkrümmung statt
- Physikalische Methoden (Jacobi-Felder) → Zugang zur internen Geometrie
- Ziel: Zusammenhang zwischen klassischer Gravitationsphysik und Quantenphysik

Garbentheorie

Schritt 7

Garbentheorie – lokale Daten, globales Bild

Garbentheorie

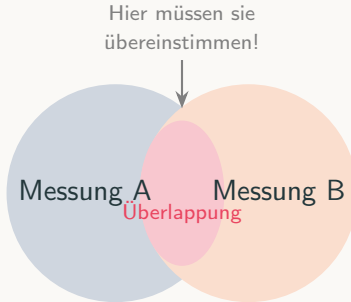
Wie fügt man lokale Beobachtungen zu einem kohärenten Ganzen?



Bildseite: Garbentheorie-Illustration

Situs – ein Ort für Messungen

- › **Situs** = etwas, an dem man Untersuchungen anstellen kann
- › Beispiele: Temperatur messen, Farbe bestimmen, Größe schätzen ...
- › Lokale Messungen → versuchen, zu einem **kohärenten Bild** zusammenzufügen



Panorama-Prinzip

- Verschiedene **Perspektiven** auf die Welt
- Realer Sachverhalt → Perspektiven **müssen überlappen**
- Überlappung = Verklebungsbedingung
- Wenn sie *nicht* überlappen → **nicht verklebbar**

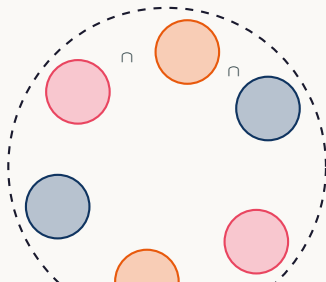
Wie beim Handy-Panorama: Bilder werden an den Überlappungen zusammengefügt.



*Bild: Handy-Panorama
mit sichtbarer Überlappung
der Einzelbilder*

Der Situs der KI

- Situs der KI = Texte von Menschen über **die Welt**
- Aber: sie sieht die Welt nur durch **kleine Schnitte** (= Worte aus Texte aus dem Training im Kontext)
- Im Internet steht “alles und das Gegenteil von allem”
→ **nicht global verklebbar**
- Aber **lokal** durchaus verklebbar → *lokale Garben*



Texte = lokale
Schnitte auf
dem Situs

Keime, Muster, Generierung

- › KI lernt auf diesen **Keimen** (lokalen Daten) Muster zu erkennen
- › Ziel: neue Keime **dynamisch generieren** können
- › Wie? Indem sie in den Raumkrümmungen **topologische Strukturen** aufbaut
- › Diese sind **approximative Muster** der realen Welt



Evidenz: Modelle lernen Geografie

- Gurnee & Tegmark (2023):
“Language Models Represent Space and Time”
- Abstände zwischen Städten (New York, Berlin, ...) in einer Dimension **realistisch** abgebildet
- Nicht explizit trainiert – emergiert als **beste Repräsentation** der Daten



*Bild: Tegmark – interne
Geografie-Karte des
Sprachmodells*

Gurnee, Tegmark. arXiv:2310.02207 (2023)

Strukturelle Kompression $\gg$ Einzelverbindungen

- **Strukturelle Kompression:**

Ein kohärentes 2D/3D-Koordinatensystem ist “billiger” als Milliarden Einzelverbindungen zwischen Städten

- **Generalisierung:**

“A östlich von B” + “B östlich von C” $\rightarrow$ automatisch auf einer Linie angeordnet

- Erlaubt Antworten auf Fragen, die **nie im Training** vorkamen

Die Daten bleiben Approximationen.

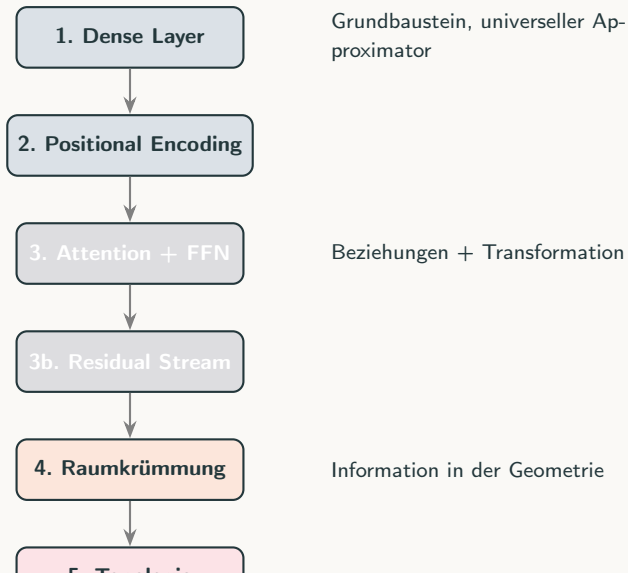
- Nicht “verstanden” (dann wäre es eine **Garbe**)
- Bleibt eine **Prä-Garbe** aus lokalen Daten
- Auch lokal nicht immer verklebbar (Widersprüche im Training)
- Von konkreter Hardware abhängig, float-Berechnungen haben nur begrenzte Genauigkeiten.
- Daher: KI kann sich irren, halluzinieren, inkonsistent sein

Zusammenfassung

Zusammenfassung

Das Gesamtbild

Der Fluss – von Tokens zu Topologie



Offene Fragen

- › Können wir mit **Jacobi-Feldern** die interne Geometrie vermessen?
- › Gibt es nachweisbare **Loop Spaces** in trainierten Netzen?
- › Wie weit reicht die **Prä-Garbe** – wo bricht sie zusammen?
- › Ist die holografische Kodierung der Schlüssel zum **mechanistischen Verständnis**?

Danke!

Fragen?